

| data-engineering / final-project

광고 이벤트 레이크하우스

만들며 한 — 고민 · 해결 · 운영 자세

발표자 신민재 · jerry04260@gmail.com

| agenda

목차

01

목표 · 무엇을 · 왜

학습 자세 · 차별점 · 아키텍처 · 왜 Iceberg

02 ★

레이어별 구현·고민

원천수집 · Kafka · Bronze · Silver · Gold

03 ★

운영 가시성

헬스체크 · 대시보드 · Slack

04 ★

실무 운영 자세

단일노드 · 장애 시나리오 · 스케일

05

지속가능성

테스트 · CI · 문서 · 협업

06

마무리

느낀점 · 아쉬움 · 향후

※ "어떻게 구성했나"보다 "레이어마다 무엇을 고민하고 어떻게 풀었나"에 집중

| 프로젝트에서 얻고 싶었던 것

단순 사용이 아니라 — 왜 · 언제 · 다음

대부분이 처음 접한 기술. "완벽히 안다"가 아니라 "제대로 이해하려" 만들었다.

- › 그냥 쓰지 않기 — 왜 이 기술인가 · 다른 방법은 없나 · 어떤 상황에서 더 유의미한가 · 다음엔 무엇을 학습할까
- › Iceberg — 왜 대용량 적재에 적합한가, 그리고 효율적 운영(백필·compaction)까지
- › 실무 관점 — 회사에서 어떻게 쓸까 · 운영이 쉬운가 · 문제를 얼마나 빨리 알아채는가

| what

무엇을 만들었나 + 차별점

광고 이벤트(실데이터 재생 + 합성) → Kafka → Spark → Iceberg 메달리온 → Trino·Superset · Airflow · Slack

01

도메인 현실성

OpenRTB 형식 · 퍼널 4토픽 ·
producer 2종

02

운영 가시성

헬스체크 · Slack 경보 · 데일리 리
포트

03

실무형 고민

장애 · 먹등 · 백필 · 스케일

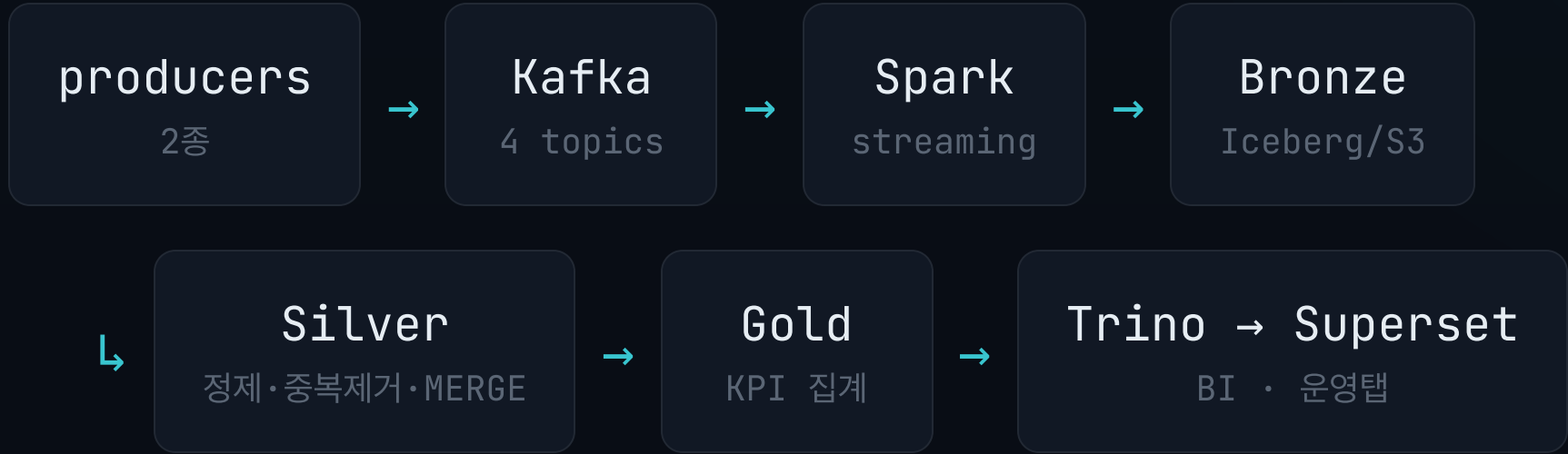
04

지속가능성

테스트 · CI · 문서 · 공통모듈

| architecture

파이프라인 아키텍처



Kafka→Bronze **스트리밍 적재** · Bronze→Gold **배치 변환** · 오케스트레이션 **Airflow** · 알림/리포트 **Slack**

| medallion

3계층 — 책임을 나눈다

BRONZE

raw 보존

파싱 없이 원본 그대로. 영구 저장소(안전지대).

SILVER

정제 · dedup

스키마 통일 · event_id 중복 제거 · 품질 검증.

GOLD

KPI 집계

campaign · banner · hourly 서빙 테이블.

각 레이어가 **한 가지 책임**만 → 고민도 해결도 그 레이어에서

| why iceberg

왜 Parquet+Glue가 아니라 Iceberg?

Parquet + Glue 의 한계

- 컬럼 변경 → 스키마 깨짐
- append만, 행 단위 update/dedup 어려움
- 롤백·스냅샷 개념 없음
- 운영 메타데이터 빈약

Iceberg 가 주는 것

- 스키마 진화 (안전한 컬럼 변경)
- ACID · 멍든 **MERGE** (dedup·백필)
- 타임트래블 · 스냅샷 롤백
- **메타테이블** → 운영 가시성

"파일 + 카탈로그"가 아니라 **테이블 포맷**이 필요했다 — dedup·백필·운영점검의 토대

| source data

원천 데이터 — 다양한 형태를 가정

실무 광고 데이터는 SDK·서버·외부 등 여러 형태로 유입 → 소스가 섞여 들어오는 상황을 가정해 2종으로 설계

- › dummy (합성) — 업계 표준 RTB(OpenRTB) 형식을 직접 써보려 구성 · 퍼널·볼륨 결정적 제어
- › criteo (실데이터) — 실제 광고 클릭·전환 로그 재생 · 진짜 분포

모든 이벤트에 **source** 필드로 출처 표시(dummy/criteo) → Silver에서 source별 파싱 분기

원천 데이터 수집

source → bronze → silver → gold

⚙ 구현

Kafka 4 topics(request·impression·click·conversion) · 메시지 key=campaign_id · producer 2종(dummy 합성 + criteo 실데이터)

💡 고민

- › RTB(OpenRTB) 형식 — 실제 광고 입찰 스키마(banner·site·device)로 현실성↑
- › 퍼널 전체 4토픽 (request 포함) — request부터 뒤야 fill_rate 등 전 퍼널 지표
- › 왜 producer 2종? — "현실 분포(criteo) + 제어 볼륨(dummy)" 둘 다. criteo엔 request/impression 없어 합성 역산

왜 굳이 Kafka인가?

처음 고민

"버퍼 역할이면 더 싸고 간단한 Redis로도 되지 않나?"

핵심 차이

Kafka는 디스크 기반 영속 이벤트 로그

재생(replay) · 컨슈머별 독립 offset · consumer group 수평확장 · 파티션 순서보장

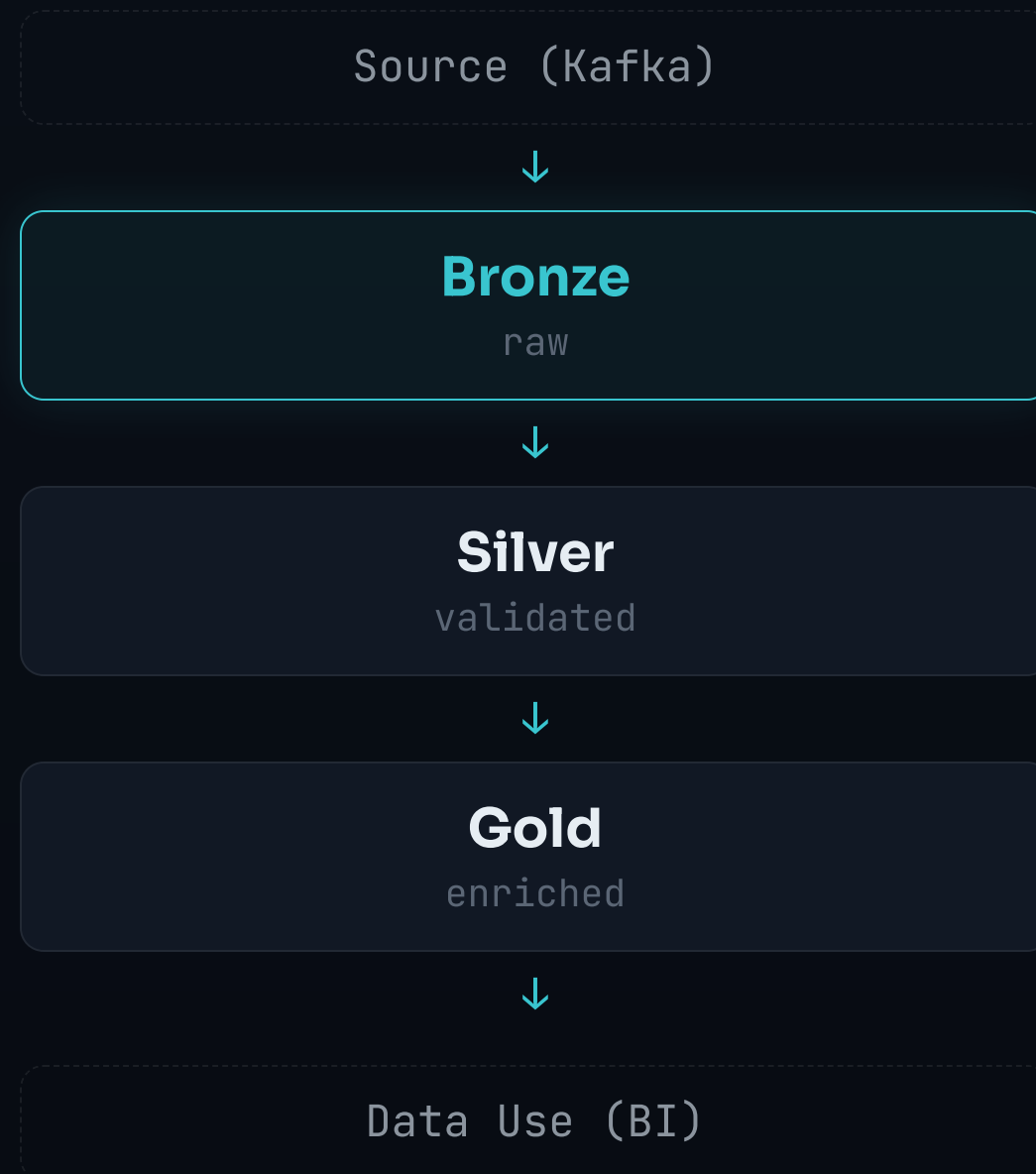
Redis 한계

메모리 중심 — persistence(RDB/AOF)로도 본질은 큐/캐시

소비 후 재읽기·대용량 스트리밍·백프레셔에 제약

필요한 건 "버퍼"가 아니라 재생 가능한 영속 로그 → Bronze 재처리·offset 복구의 전제

Medallion 설계 — Bronze



설명

- **Iceberg** 테이블 · S3 raw zone (Glue 카탈로그)
- raw JSON 그대로 보존 (파싱 X)
- 파티션 dt / hour (kafka 수신시각 기준)

역할

- Kafka 원본 이벤트 보존
- 장애·정제버그 시 **재처리 source**
- 원본 유실 방지

설계 고려

- event_time 아니라 **ingest(kafka_ts)** 기준 파티셔닝
- Spark checkpoint → kafka→bronze 스트리밍 복구
- 토픽 1:1 = 4 테이블 (append)

Bronze — 고민

중복 적재 문제

스트리밍 특성상 or 모종의 이유로 데이터 중복 적재 → **막지 않고** raw 영구보존, 중복은 Silver가 흡수

적재 단계를 단순·고속으로 유지하는 게 더 안전

왜 Bronze도 Iceberg인가? (append-only인데)

"raw zone에 Iceberg 자체를 유지할 이유가 있나?"

Iceberg 유지

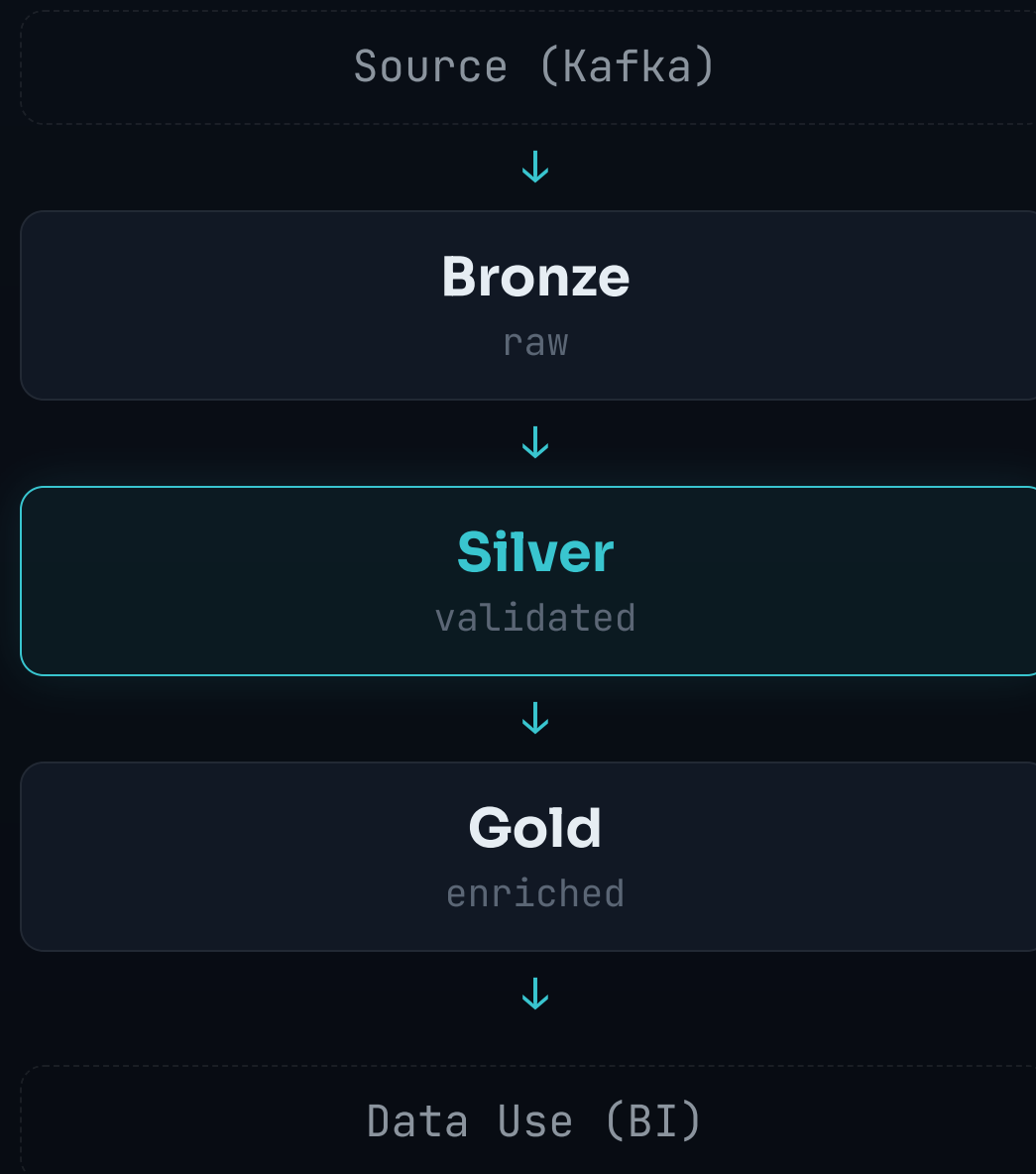
- + rewrite_data_files 자동 compaction
- + 스키마 진화 · 메타테이블 내장
- 커밋마다 snapshot·manifest 누적 → expire 필요

plain Parquet

- + 단순 · 관리 포인트↓
- + compaction은 partition overwrite로 직접
- 유지보수·메타·동시성 직접 구현

실무 Bronze는 "한 번만 읽히는" 테이블이 아니다 — 신규 Silver 백필 · 여러 Silver가 반복 read · 규모 성장 → **비용 차이 적으면 나중에 덜 번거로운 쪽**

Medallion 설계 — Silver



설명

- **Iceberg** 테이블 (COW)
- 파티션 event_date (이벤트 발생일)
- Bronze raw → 정제·통일

역할

- source별 파싱 분기 → **공통 스키마** (타입 변환)
- event_id 기반 중복 제거(먹등)
- validation → 정제 이벤트 제공

설계 고려

- 발생일 파티션 → **백필·MERGE** 대상 최소화
- MERGE upsert — 조건부로 updated_at 보존
- dedup: event_id 최신(ingested_at) 1건

Silver — 고민

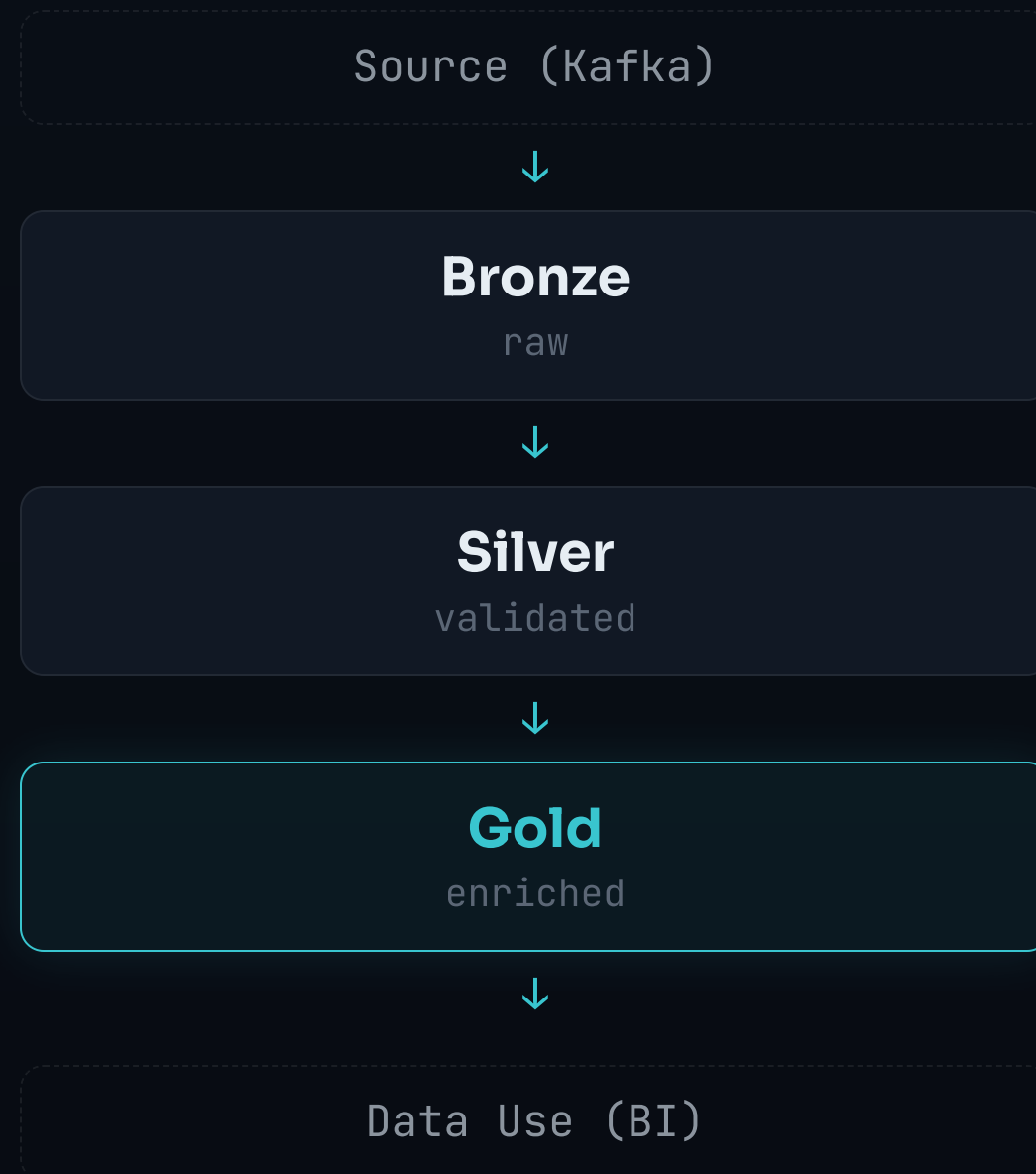
조건부 MERGE

안 바뀐 행은 UPDATE 스킵 → **updated_at** 보존 → Gold 증분이 진짜 증분

dedup ≠ 품질

중복 제거는 "두 번 안 세기" → 오염 방어는 **validation**이 따로 . 버그 시 **raw** 재처리 백필

Medallion 설계 — Gold



설명

- **Iceberg** 테이블 · 파티션 event_date
- 3 서빙 테이블 campaign_daily_stats · banner_daily_stats · hourly_funnel

역할

- KPI 집계 (campaign · banner · hourly)
- 대시보드 직결 서빙 (Trino → Superset)

설계 고려

- write= **overwritePartitions()** (바뀐 날짜만)
- updated_at 증분 → 변경 파티션만 재집계
- cost = **click의 cost(CPC)** 기준 집계

Gold — 고민

- › $\text{cost} = \text{click의 cost만 합산 (CPC)}$ — bid_price 는 입찰가일 뿐. Silver와 일치 검증
- › $\text{증분} = \text{updated_at 기준}$ — event_date 로 잡으면 criteo(2024) 통째로 누락
- › **대시보드 역설계** — 3 서빙 테이블이 KPI에 직결되도록 거꾸로 설계

왜 읽기 엔진을 둘 다? (Trino · Athena)

전제 — 쓰기(Spark)와 분리된 읽기 계층. 둘 다 읽기 전용이지만 읽기 패턴이 달라 갈라 썼다.

Athena — 임시·점검

- + 서버리스 · 인프라 0 (띄울 필요 X)
- + 스캔 바이트 과금 → 가벼운 ad-hoc·헬스 쿼리에 딱
- 반복 조회는 스캔 비용·동시성 한계

Trino — 상시 서빙

- + Superset 대시보드 뒤 상주, 반복 쿼리에 per-scan 0
- + 풀 제어(카탈로그·동시성)
- 클러스터(메모리) 직접 관리 (로컬 OOM 주의)

한 엔진에 억지로 안 몰고 읽기 패턴에 맞춰 — 점검·임시는 Athena(서버리스), 상시 대시보드는 Trino(상주)

유지보수 자동화 — 하이브리드

연산을 성격대로 나눔 — 매번 할 일(**compaction**)은 쓰기 직후, 가끔 몰아서 할 일(**expire-orphan**)은 주기로

인라인 **compaction** · **write-driven**

메달리온 DAG에서 silver·gold 쓰기 직후 압축

min-input-files=5 self-throttle(할 일 없으면 no-op) · 128MB · **partial-progress**로 OCC 처리

GC DAG · **time-driven** (04:00)

bronze compaction(스트리밍이라 주기) → **expire**(retain_last=5) → **orphan**

시간 지나 쌓인 것만 몰아서

왜 한번에 안 하고 **나눴나**

통째 DAG 한계

한 DAG가 모든 테이블 순회 → 테이블 N개면 런타임 N배

테이블 추가마다 DAG 수정 · OCC 창 매번 찾기

연산 성격이 다름

compaction은 **쓰기**에 트리거(매번) · expire-orphan은 **시간**에 트리거(가끔)

→ 매번 할 일과 가끔 할 일을 섞으면 안 됨

특히 orphan

매 쓰기마다 **전체 S3 LIST** (거의 O건인데 비용 O(전체 파일)) → 절대 인라인 X

discoverability

흩어지는 건 "로직"이 아니라 **"실행 시점"** — 로직은 한 모듈, cadence만 DAG로

cadence는 어차피 정해야 할 설계 속성이라 손해 아님

5분 안에 헬스체크가 가능한가?

헬스 지표를 세 채널로

PULL · CLI

헬스체크 스크립트

Iceberg 메타테이블 기반 8체크 자동 //

PULL · BI

대시보드 운영탭

신선도 · 행수 · 중복 · 파일 · 스냅샷

PUSH

Slack

실패 경고(@here) + 데일리 리포트 (WoW/MoM)



헬스체크 스크립트 + 대시보드

파이프라인 헬스체크 리포트

⚠️ WARN

신선도 bronze.ad_clicks 1038분 전

✅ OK

파일 silver.processed 3개·30.78MB

✅ OK

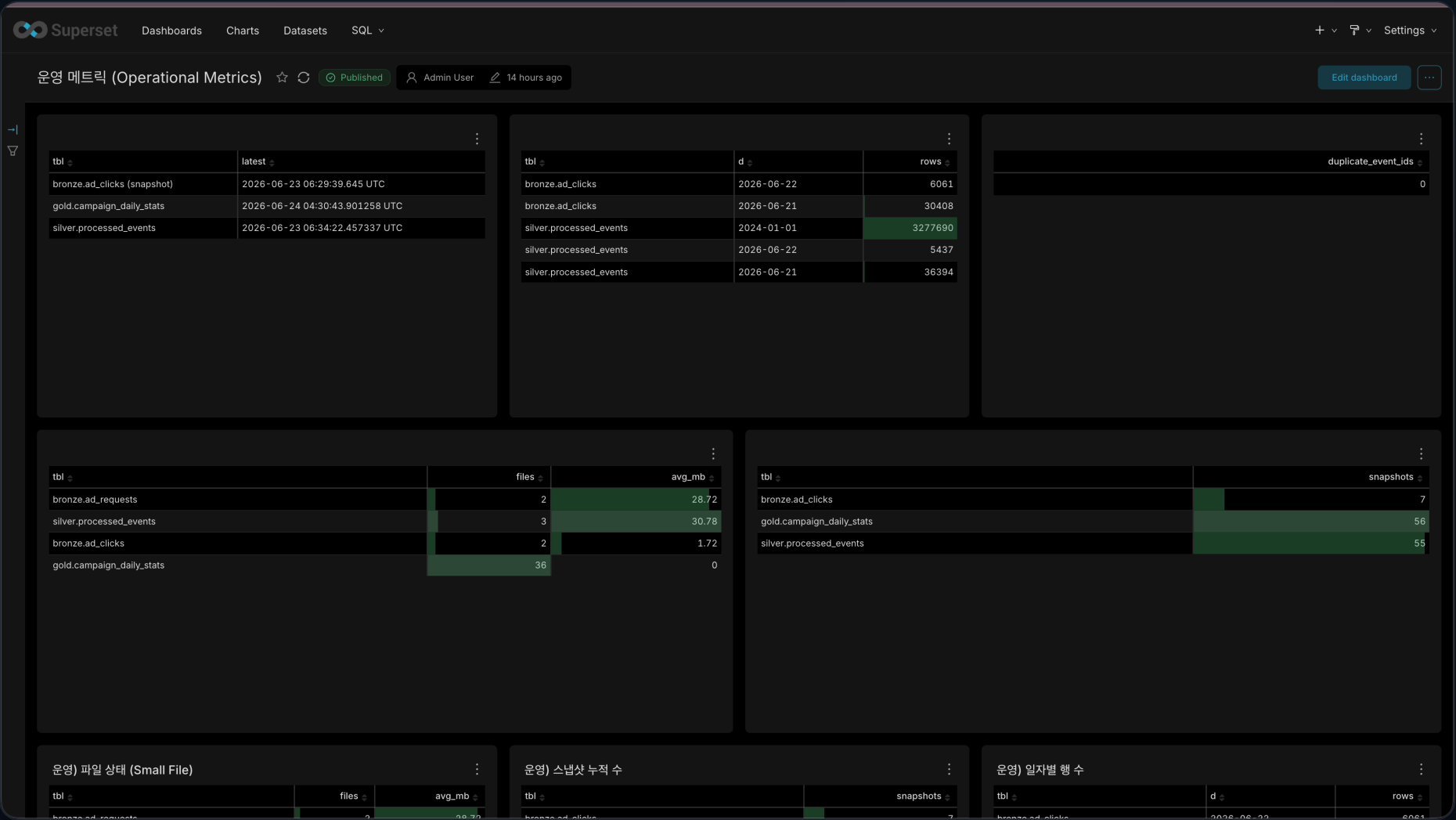
중복 event_id 0건

✅ OK

cost 정확성 차이 0.00

종합: 29체크 ❌0 ⚠️5 ✅24

① 헬스체크 스크립트 – pipeline_health.py 한 명령 · 8체크 · FAIL→exit 1 게이트



② 대시보드 운영탭 – Superset→Trino · 5차트(신선도·행수·중복·파일·스냅샷) · API 재현 번
들

이 지표들을 health_check DAG가 매시간 자동 점검 (FAIL만 경보, WARN 무음)

Slack 실패 경보



bronze-alert 앱 오후 1:57

@here 🚨 [CRITICAL] Airflow 실패 — health_check.pipeline_health (prod) 2026-06-24 13:57:34 KST

🚨 Airflow 태스크 실패 (CRITICAL)

DAG

health_check

환경

prod

실행시각

2026-06-24 13:57:34 KST

Task

pipeline_health

호스트

airflow-scheduler

Run

manual__here_test

마지막 오류

[TEST] @here 태깅 확인용 모의 실패

원인/실패 체크 확인 → [Airflow 로그 열기](#)

ad-event-lakehouse · 재시도 소진 후 알림 · WARN은 알림 없음(FAIL/크래시만)

봇이 추가한 bronze-alert

- **즉시** — health_check 잡이 실패하면 on_failure_callback이 발사
- **FAIL · 크래시만 보냄** — WARN은 무음, 노이즈 방지

데일리 KPI 리포트

- › 매일 아침 9시 — '어제 광고 어땠나'를 정상일에도 본다
- › 전주(D-7) · 전월(D-28) 동요일 — 광고는 요일 계절성이 세서 어제 대비는 의미 약함
- › 핵심 지표 + 최근 7일 표 + 30일 추세 차트

데일리 KPI 리포트 — 2026-06-23

핵심 지표 (전주·전월 동요일 대비)

지표	오늘	전주대비	전월대비
노출	265.6k	▼-4.7%	▼-3.1%
클릭	6.2k	▼-13.8%	▼-9.1%
전환	231	▼-6.9%	▼-5.3%
광고비	\$3,315	▼-15.4%	▲+0.9%
CTR	2.33%	▼-9.6%	▼-6.2%
CVR	3.74%	▲+8.1%	▲+4.1%
ROAS	0.70x	▲+10.1%	▼-6.1%

최근 7일

날짜	impr	clk	conv	cost
06-23	265.6k	6.2k	231	\$3,315
06-22	2.4k	62	4	\$49
06-21	16.0k	410	13	\$316
06-20	148.1k	3.6k	120	\$2,046
06-19	277.9k	7.1k	265	\$4,064
06-18	271.7k	6.8k	235	\$4,176
06-17	273.7k	6.3k	222	\$3,816

전주대비 = 지난주 같은 요일 대비 · 전월대비 = 4주 전 같은 요일 대비 · 첨부 = 최근 30일 추세

핵심 지표 추세 (광고비·노출·클릭·전환)

최근 30일 핵심 지표 추세 ▼



장애 시나리오 — 어떻게 대응하는가

① 새벽 OOM

Streaming 중단

checkpoint로 offset 복구 · event_id 먹등
· raw 재처리

② 로직 버그

3개월 백필

raw 보존 + 먹등 MERGE · expire/orphan이
백필 윈도우 안 가림

③ 동시 쓰기

compaction ↔ MERGE

Iceberg OCC(낙관적 동시성) · Silver/Gold
는 쓰기 직후 인라인 직렬화 · Bronze는
partial-progress 부분 커밋

장애는 일어난다는 전제로 설계 — 막는 게 아니라 복구할 수 있게

Bronze 적재 실패 — 재시작 + 경고

- 문제

실패하면 그냥 크래시 → Docker가 통째 재기동(JVM 다시, 과함)
"몇 번 시도하고 포기"라는 개념 자체가 없었다
- 재시작 루프

backoff 10→160초 · 최대 5회
10분 이상 정상 가동 시 카운터 리셋 → 연속 실패만 카운트
- Slack 경고

5회 다 소진 시에만 1번
실무 on-call 포맷(심각도·서비스·영향·조치) · Webhook(.env), WARN 무음



5회 소진 순간 1회 발사

일 100만 → 일 1억 (100x) — 어디가 먼저 깨지나

Kafka 수집

단일 브로커 → 처리량 한계·SPOF

→ 브로커·파티션 증설 + 복제, EKS면 PVC로 볼륨 durability(유실 방지)

Spark 변환

단일 worker → 100x 트래픽 못 따라감

→ executor 수평 확장 · EKS + KEDA로 Kafka lag 기반 오토스케일

Silver 쓰기

지금 **COW** → MERGE마다 파일 통째 rewrite, 100x면 비용 폭발

→ **MOR 전환**: 바뀐 것만 delete-file로 쓰고 읽을 때 merge (쓰기 가벼움)

Silver/Gold 파티션

event_date 1단(날짜만) → 하루치=1억, 파티션 과대 · small file 폭증

→ bucket(campaign) 한 겹 더(2단) 분산 · compaction 주기↑

6개월 뒤 새 팀원도 이어받게

- › 단위 테스트 + CI — 순수 로직 테스트, PR마다 GitHub Actions 자동 실행
- › 문서 · 공통모듈 — CLAUDE.md(온보딩) · spark_common/spark_defaults로 중복 제거
- › 비밀·설정 분리 — .env(gitignore) · 환경변수 주입, 코드에 키 하드코딩 X

"완벽한 일회성"보다 계속 가꿀 수 있는 구조

느낀 점 — 배운 것

- › "돌아가는 것"보다 "건강을 아는 것"이 훨씬 어렵다 —
- › Iceberg의 진짜 가치를 체감 — 스키마 진화·타임트래블·메타테이블·먹등 MERGE를 직접 써보며
- › 작은 게 정합성을 깬다 — **Spark 타임존** — Spark가 UTC로 파티셔닝 → KST 자정~오전9시가 전날 파티션에 들러 "한국 날짜" 일별 집계가 두 UTC 파티션에 걸침
- › 처음 쓰는 기술도 "왜·다른 방법·언제"를 물으면 내 것이 된다 — Kafka·Iceberg·Airflow

아쉬움 · 향후 — 남은 과제 = 다음 단계

단일 노드

Kafka 브로커1 · Spark worker1 · producer1 (로컬 한계라 **의도적 단일**)

→ EKS / Glue·EMR로 분산 전환 (env·공통모듈화로 전환 쉽게 해두려함)

타임존 정책

Bronze는 UTC raw로 뒀지만 **Silver/Gold의 KST 기준 파티셔닝**은 미결

→ "한국 날짜" 기준 파티셔닝 적용 · 더 나아가 **다른 나라 타임존(글로벌)**도 간편히 확장하는 방법 고민

스케줄 관리

스케줄 바꿀 때마다 **코드 직접 수정 + 영향 범위 수동 점검**

→ 변수·설정 기반으로 동적·중앙 관리(예: Airflow Variables)

테스트 범위

SparkSession이 무거워 **단위 테스트만** 적용

→ CI에 Spark 통합 테스트 추가

attribution

전환 기여(attribution) 분석 **미구현**

→ 클릭→전환 attribution 리포트

테이블·운영

지금은 COW만

→ 쓰기 잦은 테이블 MOR 전환 · 모니터링 스택

| thank you

감사합니다